Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 1
З дисципліни: «Програмування мобільних пристроїв на базі ОС Android»
Тема: «Основи мови Kotlin»

Виконала:
студентка групи АІ-234
Приходько С.С.
Перевірили:
Годовиченко М.А.
Соловйова Д.В.

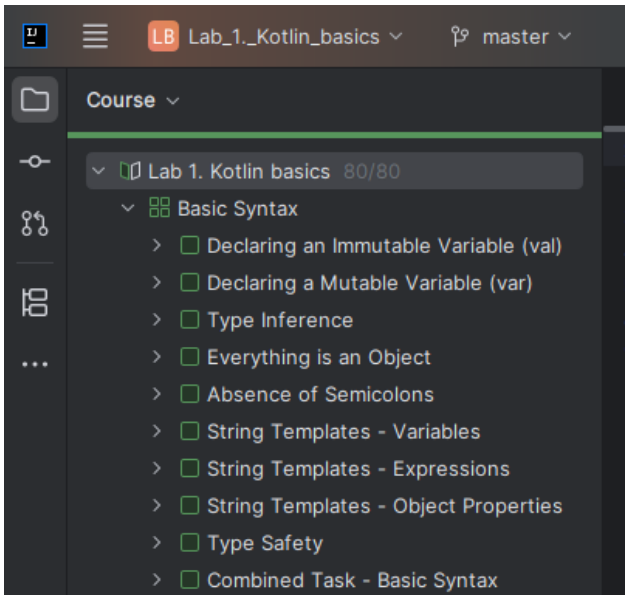
Одеса 2026

Мета роботи: Ознайомлення з основами мови Kotlin та набуття практичних навичок використання її основних конструкцій: змінних, функцій, умовних операторів, циклів, класів, колекцій, лямбда-виразів і сучасного декларативного підходу до створення інтерфейсів.

Завдання: Завантажити архів із практичними завданнями та імпортувати проєкт у середовище розробки IntelliJ IDEA. Виконати запропоновані завдання, що передбачають написання та доповнення коду мовою Kotlin у відповідних файлах.

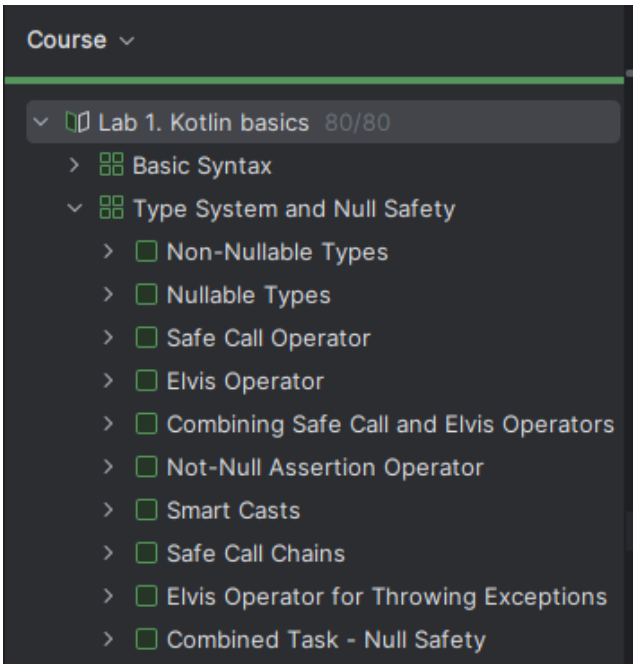
У процесі роботи необхідно реалізувати функції, класи та інші конструкції згідно з умовами задач, використовуючи основні можливості мови Kotlin. Перевірити правильність виконання завдань шляхом компіляції проєкту та проходження вбудованих тестів.

https://github.com/sofiiaprikhodko/Lab_1._Kotlin_basics.git



– У цьому розділі було ознайомлено з базовими основами синтаксису Kotlin. Компілятор Kotlin здатен самостійно виводити типи змінних завдяки механізму type inference, що дозволяє опускати явне вказування типу там, де це очевидно. Важливим аспектом є те, що в Kotlin усі типи є об'єктами, тому навіть числа мають власні методи, такі як minus(). Крапка з комою в кінці рядка не є обов'язковою. Особливу увагу було приділено рядковим шаблонам – інструменту для підстановки

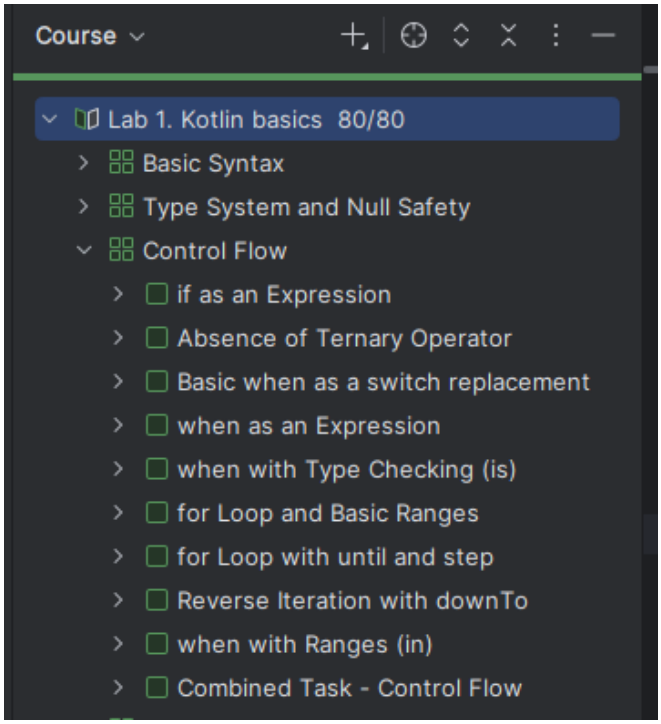
значень змінних та навіть цілих виразів у рядки за допомогою символу $ та конструкції ${}. Крім того, розглянуто різницю між змінними val (незмінні) та var (змінні), також що тип змінної, визначений один раз, не може бути змінений, що підвищує безпеку типізації. Це дозволяє писати більш зрозумілий, безпечний код, уникаючи зайвого синтаксичного шуму, характерного для багатьох інших мов програмування.



– У цьому розділі вивчено одну з ключових особливостей Kotlin – систему типів, орієнтовану на безпеку роботи з null. Головна ідея полягає в тому, що за замовчуванням усі типи є non-nullable, що дозволяє компілятору запобігти появі NullPointerException на етапі написання коду. Вивчено як явно позначати nullable типи за допомогою знаку питання (String?) та використовувати безпечний виклик ?. для доступу до методів і властивостей nullable об'єктів. Особливістю є комбінація оператора ?. з елвіс-оператором ?:, яка дозволяє задавати значення за замовчуванням або виконувати альтернативні дії у випадку null. Також розглянуто оператор !! для ситуацій, коли ми впевнені в наявності значення, хоча його використання слід мінімізувати.

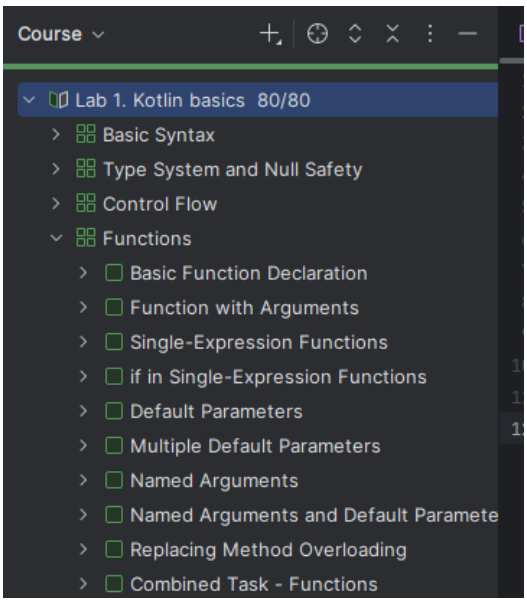
Важливим інструментом є smart cast – автоматичне приведення типу після перевірки на null, що робить код із умовними конструкціями безпечним та зручним.

Завдяки ланцюжкам безпечних викликів (user?.name?.length) ми можемо працювати з вкладеними nullable об'єктами без громіздких перевірок. У результаті ми отримали набір інструментів, які роблять код більш надійним, читабельним і позбавляють від необхідності писати зайві перевірки на null.



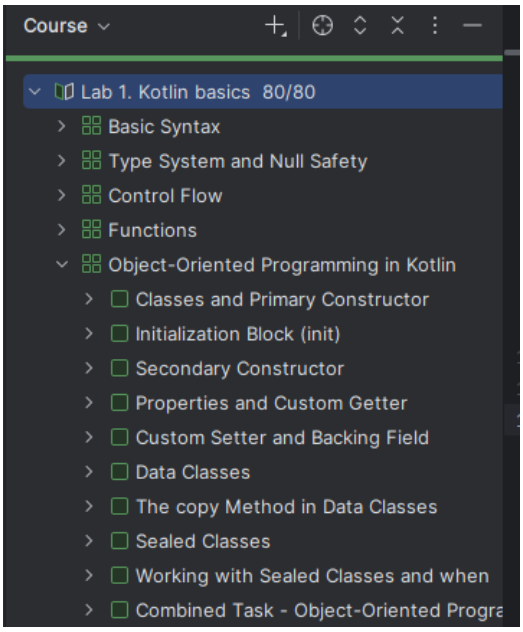
– У цьому розділі ми опанували ключові управляючі конструкції Kotlin. Головна особливість полягає в тому, що конструкції if та when у Kotlin є не просто інструкціями, а виразами, що повертають значення, що дозволяє використовувати їх безпосередньо в return або присвоєнні змінним. Вивчено як ефективно застосовувати when для різноманітних сценаріїв: від порівняння з константами до перевірки типів через is та роботи з діапазонами за допомогою оператора in.

Було розглянуто ітерацію по діапазонах з включенням обох кінців (..), виключенням верхньої межі (until) та зміною кроку (step), а також зворотну ітерацію (downTo). Завершальним прикладом стала класична задача FizzBuzz, яка продемонструвала елегантність поєднання when без аргументу з умовними перевірками у циклі. Усі ці інструменти дозволяють писати більш зрозумілий, безпечний та компактний код, уникаючи зайвих конструкцій на кшталт тернарного оператора чи громіздких switch-case.



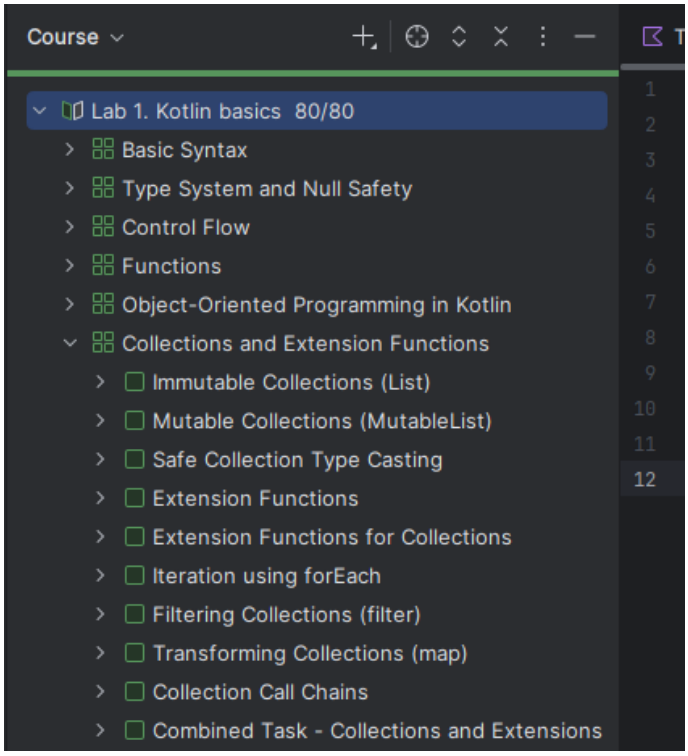
– У цьому розділі вивчено гнучкі можливості оголошення та використання функцій у Kotlin. Якщо тіло функції складається з одного виразу, ми можемо опустити фігурні дужки та ключове слово return, використовуючи знак рівності (single-expression функції), причому тип повернення компілятор виводить автоматично. Особливість становлять параметри зі значеннями за замовчуванням, які дозволяють викликати функцію з будь-якою комбінацією аргументів, усуваючи потребу в перевантаженні методів.

Також опанувано іменовані аргументи, які забезпечують читабельність коду та дозволяють змінювати порядок передачі параметрів або пропускати ті з них, що мають значення за замовчуванням.



– У цьому розділі ознайомлено з об'єктно-орієнтованим програмуванням в Kotlin. Вивчено використання первинного конструктора для оголошення властивостей безпосередньо в заголовку класу, а також блоки init для валідації вхідних даних. Важливим інструментом є вторинні конструктори, які дозволяють створювати альтернативні способи ініціалізації об'єктів.

Опанувано кастомні гетери та сетери, які дають змогу контролювати доступ до властивостей та виконувати додаткову логіку під час читання або присвоєння значень. Увагу приділено data-класам, які автоматично генерують методи equals(), hashCode(), toString() та copy(). Ключовою концепцією є sealed class, який дозволяє створювати обмежену ієрархію типів, забезпечуючи вичерпність перевірок у конструкції when.



– У цьому розділі опанувано роботу з колекціями та функціональні можливості Kotlin. Вивчено як розрізняти незмінні колекції (List) та змінні (MutableList), що є важливим аспектом безпеки даних. Головну увагу було приділено функціям-розширенням, які дозволяють додавати нову поведінку до існуючих класів без успадкування, що відкриває широкі можливості для створення зручних API.

Важливість становлять функції вищого порядку для роботи з колекціями: forEach для ітерації, filter для відбору елементів за умовою та map для трансформації даних. Їхнє поєднання в ланцюжки викликів дозволяє виконувати складні операції над даними в декларативному стилі, уникаючи громіздких циклів та тимчасових змінних.

– У цьому розділі опановано принципи створення декларативного інтерфейсу в Kotlin. Головна ідея полягає у використанні функцій верхнього рівня з PascalCase-назвами, які візуально нагадують класи, що робить ієрархію компонентів чистою та зрозумілою. Вивчено як ефективно використовувати іменовані аргументи для конфігурації компонентів, що дозволяє пропускати параметри зі значеннями за замовчуванням та змінювати порядок передачі аргументів.

Також ознайомлено з trailing lambda – якщо останнім параметром функції є лямбда, її можна винести за межі круглих дужок, а при єдиному лямбда-параметрі

– взагалі опустити дужки, створюючи природний DSL-синтаксис. Вивчено лямбди з одержувачем, які дозволяють обмежувати доступні методи залежно від контексту, що робить API типобезпечним та зрозумілим.

Висновки: У ході виконання лабораторної роботи було вивчено ключові можливості мови Kotlin, зокрема роботу з типами даних, null-безпекою, функціями, умовними конструкціями та циклами. Також було розглянуто принципи об’єктно-орієнтованого програмування, включаючи створення класів, використання data та sealed класів, а також механізми інкапсуляції через гетери і сетери. Приділено увагу роботі з колекціями, функціям-розширенням і лямбда-виразам, що дозволяють писати більш компактний і зрозумілий код. Крім того, було освоєно основи декларативного підходу до побудови інтерфейсів, характерним для сучасної розробки на Kotlin. У результаті роботи було закріплено теоретичні знання та сформовано практичні навички написання коду.